



01-DARS · 14 TADAN

Web qanday ishlaydi

Internet, mijoz-server modeli, DNS, HTTP protokoli, so'rov va javoblar, status kodlari hamda brauzer sahifani qanday chizishi.

Boshlang'ich · ~60 daqiqa

Kod yozishni boshlashdan oldin eng muhim savolga javob topamiz: **brauzerga** `google.com` **deb yozib** `Enter` **bosganingizda aslida nima sodir bo'ladi?** Bu savolga chuqur javob bera olgan dasturchi frontend, backend, tarmoq va xavfsizlik muammolarining aksariyatini tez tushunadi. Ushbu dars — qolgan barcha darslarning poydevori.

Internet va Web — farqi nimada?

Ko'pchilik bu ikki so'zni sinonim deb o'ylaydi, lekin ular turli narsalar:

- **Internet** — butun dunyodagi kompyuterlarni bir-biriga bog'lovchi *jismoniy va mantiqiy tarmoq* (kabellar, routerlar, Wi-Fi, mobil tarmoqlar, TCP/IP protokollari). U 1960–70-yillarda ARPANET loyihasidan boshlangan.
- **Web (World Wide Web)** — Internet *ustida* ishlaydigan xizmatlardan biri: bir-biriga havolalar orqali bog'langan hujjatlar (sahifalar) tizimi. Uni 1989-yilda Tim Berners-Li CERN'da ixtiro qilgan va uchta asosiy texnologiyani yaratgan: **HTML** (hujjat tili), **URL** (manzil) va **HTTP** (uzatish protokoli).

Internet ustida boshqa xizmatlar ham bor: elektron pochta (SMTP), fayl uzatish (FTP), onlayn o'yinlar, video qo'ng'iroqlar va hokazo. Web — ulardan faqat bittasi, lekin eng mashhuri.

Ma'lumot tarmoqda qanday harakatlanadi?

Siz yuborgan har qanday ma'lumot (rasm, matn, video) kichik bo'laklarga — **paketlarga** bo'linadi. Har bir paketda qabul qiluvchining **IP-manzili** yoziladi va paketlar routerlar orqali turli yo'llar bilan manzilga yetib boradi, u yerda yana birlashtiriladi. Bu jarayonni protokollar qatlamlari boshqaradi:

Qatlam	Protokol misollari	Vazifasi

Ilova (Application)	HTTP, HTTPS, DNS, SMTP, WebSocket	Dasturlar nima haqida "gaplashishini" belgilaydi
Transport	TCP, UDP	Ma'lumot to'liq va to'g'ri tartibda yetib borishini kafolatlaydi (TCP)
Tarmoq (Internet)	IP (IPv4, IPv6)	Paketni qaysi kompyuterga yetkazishni belgilaydi
Kanal (Link)	Ethernet, Wi-Fi	Ma'lumotni jismoniy muhit orqali uzatadi



Veb-dasturchi uchun nima muhim?

Siz asosan eng yuqori qatlamda — **HTTP** bilan ishlaysiz. Lekin TCP ulanish o'rnatish vaqt olishini, IP-manzil nima ekanini va DNS qanday ishlashini bilish sahifa sekin yuklanishi yoki "server topilmadi" kabi muammolarni tushunishda juda yordam beradi.

Mijoz-server (Client-Server) modeli

Web **mijoz-server** arxitekturasiga asoslangan. Bunda ikki tomon bor:

- **Mijoz (client)** — so'rov yuboruvchi dastur. Ko'pincha bu brauzer, lekin mobil ilova, `curl` buyrug'i yoki boshqa server ham mijoz bo'lishi mumkin.
- **Server** — so'rovlarni kutib turadigan va ularga javob qaytaradigan dastur (va u ishlayotgan kompyuter). Server 24/7 yoqilgan bo'ladi va bir vaqtda minglab mijozlarga xizmat qiladi.



Mijoz (brauzer)

— 1. So'rov (Request) →



Server

← 2. Javob (Response) —



Mijoz

Muloqotni har doim mijoz boshlaydi; server faqat javob beradi

Bu modelning muhim xususiyati: **muloqotni doim mijoz boshlaydi**. Server o'zi mijozga "birinchi bo'lib" xabar yubora olmaydi (oddiy HTTP'da). Real vaqtdagi chatlar uchun WebSocket kabi qo'shimcha texnologiyalar ishlatilishining sababi ham shu.

Statik va dinamik sahifalar

- **Statik sayt** — server diskdagi tayyor fayllarni (`.html` , `.css` , rasmlar) o'zgartirmasdan yuboradi. Har bir foydalanuvchi bir xil sahifani ko'radi. Masalan: portfolio, hujjatlar sayti (aynan siz o'qiyotgan ushbu qo'llanma ham statik sayt).
- **Dinamik sayt** — server har bir so'rov uchun javobni dastur orqali *hosil qiladi*: ma'lumotlar bazasiga murojaat qiladi, foydalanuvchini aniqlaydi, hisob-kitob qiladi. Masalan: Instagram lentasi har bir odam uchun boshqacha.

URL tuzilishi

URL (Uniform Resource Locator) — internetdagi resursning to'liq manzili. Uni qismlarga ajratib ko'ramiz:

Matn

`https://shop.example.uz:443/products/phones?brand=samsung&sort=price#reviews`

schema

host

port

yo'l (path)

so'rov satri (query)

fragment

Qism	Misol	Ma'nosi
Sxema (protocol)	<code>https</code>	Qaysi protokol bilan ulanish. <code>https</code> = shifrlangan HTTP
Host (domen)	<code>shop.example.uz</code>	Server nomi. <code>shop</code> — subdomen, <code>example</code> — domen, <code>uz</code> — yuqori darajali domen (TLD)
Port	<code>443</code>	Serverdagi "eshik" raqami. HTTP uchun standart 80, HTTPS uchun 443 (yozilmasa ham shu nazarda tutiladi)
Yo'l (path)	<code>/products/phones</code>	Server ichidagi aniq resurs
Query	<code>?brand=samsung&sort=price</code>	Qo'shimcha parametrlar: <code>kalit=qiymat</code> juftliklari, <code>&</code> bilan ajratiladi
Fragment	<code>#reviews</code>	Sahifa ichidagi bo'lim. Serverga yuborilmaydi , faqat brauzerda ishlaydi

Full-stack qo'llanma · 01. Web qanday ishlaydi

3 / 16



URL kodlash (encoding)

URL'da bo'sh joy, kirill harflari yoki maxsus belgilar bevosita yozilmaydi — ular `%` bilan kodlanadi. Masalan, bo'sh joy `%20` ga, `o'` esa `o%27` ga aylanadi. JavaScript'da buning uchun `encodeURIComponent("sa'lon dunyo")` funksiyasi bor.

DNS — internetning telefon kitobi

Kompyuterlar bir-birini **IP-manzil** orqali topadi (masalan `142.250.74.46`). Odamlar uchun esa `google.com` kabi nomlarni eslab qolish oson. **DNS (Domain Name System)** — domen nomini IP-manzilga aylantiruvchi tizim.

DNS so'rovi bosqichma-bosqich

- Brauzer keshi** — brauzer bu domenni yaqinda ko'rganmi? Ha bo'lsa, tayyor IP ishlatiladi.
- Operatsion tizim keshi** va `hosts` fayli tekshiriladi.
- Rekursiv resolver** — odatda internet provayderingiz yoki `8.8.8.8` (Google), `1.1.1.1` (Cloudflare) serveri. U javobni topish uchun quyidagilardan so'raydi:
- Root server** → "`.com` zonasi bilan qaysi server shug'ullanadi?"
- TLD server** (`.com`) → "`google.com` uchun mas'ul (authoritative) server qaysi?"
- Authoritative server** → "`google.com` ning IP-manzili: `142.250.74.46`".
- Javob keshlanadi (TTL — "yashash vaqti" davomida) va brauzerga qaytariladi.

Asosiy DNS yozuvlari

Yozuv	Vazifasi	Misol
<code>A</code>	Domen → IPv4 manzil	<code>example.uz</code> → <code>93.184.216.34</code>
<code>AAAA</code>	Domen → IPv6 manzil	<code>example.uz</code> → <code>2606:2800:220:1::</code>
<code>CNAME</code>	Domen → boshqa domen (taxallus)	<code>www.example.uz</code> → <code>example.uz</code>
<code>MX</code>	Pochta serveri	<code>example.uz</code> → <code>mail.example.uz</code>

TXT

Ixtiyoriy matn (domen egaligini tasdiqlash
va h.k.)

"v=spf1 include:_spf.google.com
~all"

Sayingizni hostingga joylaganingizda (14-dars) aynan shu yozuvlarni domen sozlamalarida kiritasiz.

HTTP protokoli chuqur

HTTP (HyperText Transfer Protocol) — mijoz va server o'rtasidagi muloqot qoidalari. U **matnli** (HTTP/1.1 da) va **holatsiz (stateless)** protokol: har bir so'rov mustaqil, server oldingi so'rovni "eslamaydi". Foydalanuvchi tizimga kirganini eslab qolish uchun cookie va tokenlar ishlatiladi (11-dars).

HTTP so'rovi (Request) tuzilishi

HTTP

```
POST /api/users HTTP/1.1
Host: api.example.uz
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: application/json
Content-Length: 46

{"name": "Aziza", "email": "aziza@mail.uz"}
```

1. **Boshlang'ich qator:** *metod* (`POST`), *yo'l* (`/api/users`), *versiya* (`HTTP/1.1`).
2. **Sarlavhalar (headers):** `Nom: qiymat` ko'rinishidagi meta-ma'lumotlar.
3. **Bo'sh qator** — sarlavhalar tugaganini bildiradi.
4. **Tana (body)** — yuborilayotgan ma'lumot (GET so'rovlarida odatda bo'lmaydi).

HTTP javobi (Response) tuzilishi

```
HTTP

HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Location: /api/users/57
Set-Cookie: session=abc123; HttpOnly; Secure
Cache-Control: no-store
Date: Sat, 03 Oct 2026 10:15:00 GMT

{"id": 57, "name": "Aziza", "email": "aziza@mail.uz"}
```

HTTP metodlari

Metod	Ma'nosi	Tana bormi?	Xavfsiz?	Idempotent?
GET	Resursni o'qish	Yo'q	Ha	Ha
POST	Yangi resurs yaratish / amal bajarish	Ha	Yo'q	Yo'q
PUT	Resursni to'liq almashtirish	Ha	Yo'q	Ha
PATCH	Resursni qisman yangilash	Ha	Yo'q	Odatda yo'q
DELETE	Resursni o'chirish	Odatda yo'q	Yo'q	Ha
HEAD	GET kabi, lekin faqat sarlavhalar	Yo'q	Ha	Ha
OPTIONS	Server qaysi metodlarni qabul qilishini so'rash (CORS)	Yo'q	Ha	Ha

- **Xavfsiz (safe)** — serverdagi ma'lumotni o'zgartirmaydi.
- **Idempotent** — bir xil so'rovni 1 marta yoki 10 marta yuborish natijasi bir xil. Masalan, `DELETE /users/5` ni ikki marta yuborsangiz ham foydalanuvchi bir marta o'chadi. `POST /orders` ni ikki marta yuborsangiz esa ikkita buyurtma yaratiladi — shuning uchun "To'lash" tugmasini ikki marta bosish xavfli.

Status kodlari

Server javobining birinchi qatoridagi uch xonali raqam so'rov natijasini bildiradi. Birinchi raqam — guruh:

Kod	Ma'nosi	Qachon ishlatiladi
2xx — Muvaffaqiyat		
200 OK	Hammasi joyida	GET, PUT, PATCH muvaffaqiyatli
201 Created	Yangi resurs yaratildi	POST orqali foydalanuvchi qo'shildi
204 No Content	Bajarildi, qaytaradigan narsa yo'q	DELETE muvaffaqiyatli
3xx — Yo'naltirish		
301 Moved Permanently	Manzil butunlay o'zgargan	http:// → https://
302 Found	Vaqtincha boshqa manzil	Login'dan keyin bosh sahifaga
304 Not Modified	Keshdagi nusxa hali yangi	Brauzer faylni qayta yuklamaydi
4xx — Mijoz xatosi		
400 Bad Request	So'rov noto'g'ri	Email maydoni bo'sh yuborildi
401 Unauthorized	Tizimga kirilmagan	Token yo'q yoki eskirgan
403 Forbidden	Kirilgan, lekin ruxsat yo'q	Oddiy foydalanuvchi admin sahifasiga
404 Not Found	Resurs topilmadi	/users/9999 mavjud emas
409 Conflict	Holat ziddiyati	Bu email allaqachon ro'yxatdan o'tgan
422 Unprocessable Entity	Ma'lumot validatsiyadan o'tmadi	Parol juda qisqa

429 Too Many Requests	Juda ko'p so'rov	Rate limit oshib ketdi
5xx — Server xatosi		
500 Internal Server Error	Serverda kutilmagan xato	Kodda bug
502 Bad Gateway	Proksi orqadagi serverdan javob ololmadi	Node.js ilova o'chib qolgan, Nginx ishlayapti
503 Service Unavailable	Server vaqtincha band	Texnik ishlar, haddan tashqari yuklama



Eslab qolish uchun

4xx — "sen xato qilding" (mijoz), 5xx — "men xato qildim" (server). API yozganda to'g'ri status kod qaytarish — professional backend dasturchining belgisi.

Muhim sarlavhalar (headers)

Sarlavha	Yo'nalish	Vazifasi
Content-Type	Ikkalasi	Tana formati: text/html, application/json, multipart/form-data
Authorization	So'rov	Kimligingizni isbotlash: Bearer <token>
Cookie / Set-Cookie	So'rov / Javob	Brauzerda kichik ma'lumot saqlash (sessiya)
Cache-Control	Javob	Javobni qancha vaqt keshlash mumkin
Location	Javob	Yo'naltirish manzili yoki yangi resurs manzili
Access-Control-Allow-Origin	Javob	CORS: qaysi saytlar bu API'ga murojaat qila oladi
User-Agent	So'rov	Mijoz dasturi haqida ma'lumot

HTTPS va TLS

Oddiy HTTP'da ma'lumot **ochiq matn** sifatida uzatiladi — Wi-Fi tarmog'idagi har qanday odam parolingizni o'qiy oladi. **HTTPS** = HTTP + **TLS** shifrlash. TLS uchta narsani kafolatlaydi:

- **Maxfiylik** — ma'lumotni yo'lda hech kim o'qiy olmaydi;
- **Yaxlitlik** — ma'lumotni yo'lda hech kim o'zgartira olmaydi;
- **Haqiqiylik** — siz haqiqatan `bank.uz` bilan gaplashyapsiz (SSL sertifikat orqali).

Bugungi kunda har qanday sayt HTTPS'da bo'lishi shart. Let's Encrypt bepul sertifikat beradi.

HTTP versiyalari

- **HTTP/1.1** (1997) — matnli, bitta ulanishda so'rovlar ketma-ket ketadi.
- **HTTP/2** (2015) — ikkilik format, bitta ulanishda ko'plab so'rovlar parallel (multiplexing), sarlavhalar siqiladi.
- **HTTP/3** (2022) — TCP o'rniga QUIC (UDP asosida), mobil tarmoqlarda tezroq.

Dasturchi sifatida siz yozadigan kod deyarli o'zgarmaydi — metodlar, status kodlar va sarlavhalar ma'nosi bir xil.

Brauzer sahifani qanday chizadi?

Server HTML qaytargandan keyin brauzer ichida murakkab jarayon boshlanadi — buni **Critical Rendering Path** deyiladi:

1. **HTML parsing → DOM**. Brauzer HTML matnini o'qib, teglar daraxtini — **DOM (Document Object Model)** ni quradi.
2. **CSS parsing → CSSOM**. Barcha CSS qoidalaridan uslublar daraxti quriladi.
3. **Render tree**. DOM va CSSOM birlashtiriladi; ko'rinmas elementlar (`display:none` , `<head>`) tashlab yuboriladi.
4. **Layout (reflow)**. Har bir elementning ekrandagi aniq o'lchami va joylashuvi hisoblanadi.
5. **Paint**. Piksellar chiziladi: ranglar, matn, soyalar, rasmlar.
6. **Composite**. Qatlamlar birlashtirilib ekranga chiqariladi.



Nega bu muhim?

`<script>` tegi HTML parsing'ni **to'xtatib qo'yadi** — brauzer skriptni yuklab, bajarib bo'lgunicha qolgan HTML'ni o'qimaydi. Shuning uchun skriptlarni `<body>` oxiriga qo'yish yoki `defer` atributidan foydalanish tavsiya qilinadi. CSS esa chizishni bloklaydi — shuning uchun u `<head>` ichida bo'ladi.

To'liq zanjir: Enter bosilgandan sahifa chiqqungacha

1. Brauzer URL'ni tahlil qiladi.
2. **DNS** orqali domen IP-manzilga aylantiriladi.
3. Server bilan **TCP ulanish** o'rnatiladi (3 bosqichli "qo'l siqish": SYN → SYN-ACK → ACK).
4. HTTPS bo'lsa — **TLS handshake**: shifrlash kalitlari kelishiladi, sertifikat tekshiriladi.
5. **HTTP so'rov** yuboriladi: `GET / HTTP/1.1`.
6. Server so'rovni qayta ishlaydi (balki bazaga murojaat qiladi) va **HTML javob** qaytaradi.
7. Brauzer HTML'ni o'qiy boshlaydi, unda CSS, JS, rasm havolalarini ko'rib, ular uchun **qo'shimcha so'rovlar** yuboradi.
8. DOM + CSSOM → Layout → Paint → sahifa ekranda.
9. JavaScript bajarilib, sahifa interaktiv bo'ladi.

Frontend, Backend va Full-stack

	Frontend	Backend
Qayerda ishlaydi	Foydalanuvchi brauzerida	Serverda
Texnologiyalar	HTML, CSS, JavaScript, React/Vue/Angular	Node.js, Python, PHP, Go, Java + SQL/NoSQL bazalar
Vazifasi	Interfeys, animatsiya, foydalanuvchi bilan muloqot	Ma'lumot saqlash, biznes-mantiq, xavfsizlik, API
Kodni kim ko'radi	Har kim (DevTools orqali)	Faqat siz



Oltin qoida

Frontend kodiga **hech qachon ishonmang**. Foydalanuvchi brauzerdagi JavaScript'ni o'zgartirishi, tekshiruvlarni chetlab o'tishi va serverga istalgan so'rovni yuborishi mumkin. Narx, ruxsatlar, validatsiya — barchasi **serverda** qayta tekshirilishi shart.

Amaliy misollar

MISOL 1

DevTools orqali real saytning so'rovlarini tahlil qilish

Har bir veb-dasturchining eng yaqin do'sti — brauzerning **Developer Tools** paneli.

1. Chrome'da istalgan saytni oching (masalan, `https://kun.uz`).
2. `F12` (yoki `Ctrl + Shift + I`) bosib va **Network** yorlig'iga o'ting.
3. Sahifani `Ctrl + R` bilan yangilang.
4. Ro'yxatdagi birinchi so'rovni (sahifaning o'zi, turi `document`) bosib.

Siz quyidagilarni ko'rasiz:

- **Headers** yorlig'i — Request URL, Request Method (`GET`), Status Code (`200`), Remote Address (serverning IP-manzili va porti), barcha so'rov va javob sarlavhalari.
- **Response** — server qaytargan xom HTML.
- **Timing** — vaqt qayerga ketgani: DNS Lookup, Initial connection (TCP), SSL, Waiting for server response (TTFB), Content Download.

Pastki satrda jami so'rovlar soni (odatda 50–150 ta!), yuklangan hajm va **DOMContentLoaded / Load** vaqtlari ko'rsatiladi. **Type** ustuni bo'yicha saralab, sahifa nechta rasm, skript va CSS yuklayotganini ko'ring.



"Disable cache" belgisini qo'yib, sahifani qayta yuklang — keshsiz yuklanish qancha uzoq davom etishini taqqoslang. Network panelidagi "Throttling" orqali sekin 3G internetni ham sinab ko'rish mumkin.

`curl` — terminaldan HTTP so'rov yuboruvchi dastur. U Windows 10+, macOS va Linux'da o'rnatilgan bo'ladi. Brauzer "yashiradigan" narsalarni ko'rishning eng yaxshi usuli.

Terminal

```
# 1. Oddiy GET so'rov – javob tanasini chiqaradi
curl https://jsonplaceholder.typicode.com/users/1

# 2. -i : javob sarlavhalarini ham ko'rsatish
curl -i https://jsonplaceholder.typicode.com/users/1

# 3. -v : to'liq jarayon (DNS, TCP, TLS, so'rov va javob sarlavhalari)
curl -v https://example.com 2>&1 | head -40

# 4. Yo'naltirishni ko'rish (301) va unga ergashish (-L)
curl -I http://github.com
curl -IL http://github.com

# 5. POST so'rov: JSON yuborish
curl -X POST https://jsonplaceholder.typicode.com/posts \
  -H "Content-Type: application/json" \
  -d '{"title": "Salom", "body": "Birinchi post", "userId": 1}'

# 6. Mavjud bo'lmagan resurs – 404 ni ko'rish
curl -i https://jsonplaceholder.typicode.com/users/9999
```

5-buyruq natijasi:

JSON

```
{
  "title": "Salom",
  "body": "Birinchi post",
  "userId": 1,
  "id": 101
}
```

E'tibor bering: server `201 Created` qaytardi va yangi resursga `id` berdi. Aynan shunday API'ni siz 9-darsda o'zingiz yozasiz.

MISOL 3 DNS so'rovini qo'lda tekshirish

Terminal

```
# Domen IP-manzilini bilish (Windows, macOS, Linux)
nslookup github.com

# Batafsil (macOS / Linux): A, MX, TXT yozuvlari
dig github.com A +short
dig gmail.com MX +short
dig google.com TXT +short

# Ma'lum DNS server orqali so'rash (Cloudflare 1.1.1.1)
nslookup kun.uz 1.1.1.1

# Paket serverga qaysi routerlar orqali borayotganini ko'rish
tracert google.com      # macOS / Linux
tracert google.com      # Windows
```

Tajriba: o'z kompyuteringizdagi `hosts` faylini oching (Windows:

`C:\Windows\System32\drivers\etc\hosts`, macOS/Linux: `/etc/hosts`) va administrator huquqi bilan quyidagi qatorni qo'shing:

Matn

```
127.0.0.1   mening-saytim.test
```

Endi brauzerda `http://mening-saytim.test` manzili sizning o'z kompyuteringizga (`localhost`) yo'naladi — chunki `hosts` fayli DNS'dan oldin tekshiriladi. Tajribadan keyin qatorni o'chirib qo'ying.

MISOL 4 20 qatorda o'z HTTP serveringizni yozish

Server — bu sehr emas, shunchaki portni tinglab turuvchi dastur. Buni isbotlash uchun Node.js'ning o'rnatilgan `http` moduli bilan oddiy server yozamiz (Node.js 8-darsda batafsil o'rganiladi; hozir faqat o'rnatish).

server.js

```
const http = require('http');

const server = http.createServer((req, res) => {
  // Har bir kelgan so'rovni terminalga chiqaramiz
  console.log(`${req.method} ${req.url}`);
  console.log('Sarlavhalar:', req.headers);

  if (req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
    res.end('<h1>Salom! Bu mening birinchi serverim</h1><a  
href="/api/vaqt">Vaqtni bilish</a>');
  } else if (req.url === '/api/vaqt') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ vaqt: new Date().toISOString() }));
  } else {
    res.writeHead(404, { 'Content-Type': 'text/plain; charset=utf-8' });
    res.end('404 - Sahifa topilmadi');
  }
});

server.listen(3000, () => console.log('Server http://localhost:3000 da  
ishlayapti'));
```

Ishga tushirish:

Terminal

```
node server.js
# Boshqa terminalda:
curl -i http://localhost:3000/api/vaqt
curl -i http://localhost:3000/yoq-sahifa
```

Brauzerda `http://localhost:3000` ni oching va server terminaliga qarang: siz bitta sahifa ochgan bo'lsangiz ham, brauzer `/favicon.ico` uchun ham avtomatik so'rov yuborganini ko'rasiz. Endi siz mijoz (brauzer) va server o'rtasidagi muloqotni ikkala tomondan ham ko'radingiz.

Mashqlar



Mustaqil ish

1. `https://www.youtube.com/watch?v=dQw4w9WgXcQ&t=42s` URL'ini qismlarga ajrating: sxema, host, subdomen, yo'l, query parametrlari.
2. DevTools orqali uchta turli saytda nechta so'rov yuborilishini va sahifa necha soniyada yuklanishini solishtiring. Eng ko'p joy egallagan fayl turi qaysi?
3. `curl -i` orqali `http://google.com` ga so'rov yuboring. Qanday status kod va `Location` sarlavhasi qaytdi? Nega?
4. 4-misoldagi serverga `/api/salom?ism=Ali` marshrutini qo'shing, u `{"xabar": "Salom, Ali!"}` qaytarsin. (Maslahat: `new URL(req.url, 'http://localhost').searchParams.get('ism')`)
5. Quyidagi holatlar uchun to'g'ri status kodni tanlang: (a) parol noto'g'ri; (b) foydalanuvchi boshqa birovning buyurtmasini o'chirmoqchi; (c) yangi maqola yaratildi; (d) ma'lumotlar bazasi bilan aloqa uzildi.

▼ 5-savol javobi

(a) `401` ; (b) `403` (yoki ma'lumotni yashirish uchun `404`); (c) `201` ; (d) `500` yoki `503` .



Xulosa

- Internet — tarmoq; Web — uning ustidagi HTML + URL + HTTP tizimi.
- Mijoz so'rov yuboradi, server javob qaytaradi; muloqotni doim mijoz boshlaydi.
- DNS domen nomini IP-manzilga aylantiradi; natijalar keshlanadi.
- HTTP so'rovi: metod + yo'l + sarlavhalar + tana. Javob: status kod + sarlavhalar + tana.
- Status kodlar: 2xx muvaffaqiyat, 3xx yo'naltirish, 4xx mijoz xatosi, 5xx server xatosi.
- Brauzer HTML → DOM, CSS → CSSOM, keyin Layout va Paint orqali sahifani chizadi.

- Serverda doim qayta tekshiring — frontendga ishonib bo'lmaydi.

☐ Darsni o'qib bo'ldim